AIM :
To port FreeRTOS to an Arduino UNO or STM32 development board and create multiple tasks to understand task scheduling and multitasking in a Real-Time Operating System (RTOS).

THEORY : FreeRTOS is a light operating system made for small computers and systems that are built into bigger machines.Unlike regular programs that do one thing after another, FreeRTOS lets you split your work into parts called tasks.These tasks can all use the computer's processor at different times.

In this test, FreeRTOS is set up on a small computer board like an Arduino UNO or an STM32 board.
We make several tasks to see how FreeRTOS handles the computer's time between them.

Each task is like a separate job.
One task might turn an LED on or off, another might read information from a sensor, and another might show data on a screen.FreeRTOS has a special tool called a scheduler that decides which task gets to use the processor at any moment.

The scheduler uses priorities to choose which task gets the processor.
Tasks can also be in different conditions like running, waiting, stopped, or paused.If a task waits for a certain time using a function like vTaskDelay(), it stops running for that time, so another task can use the processor.

The main things we show in this test are:

- Task creation: Regular functions are turned into tasks for FreeRTOS.
- Task scheduling: The scheduler picks which tasks to run next.
- Multitasking: It looks like many tasks are running at once because they share the computer's time.
- Task priority: Tasks with higher priority get to run before those with lower priority.
- Task delay: A task can stop itself for a specific amount of time.
- Context switching: The computer moves from one task to another based on the scheduler.
- Tick interrupt: FreeRTOS uses a timer that ticks at regular intervals to keep track of time and manage tasks.

ALGORITHM :

- Choose the development board you want to use, like an Arduino UNO or an STM32 board.

- Prepare the development environment and install the right FreeRTOS library for that board.
- Set up the board and check that a simple program can be built and sent to it.
- Add the FreeRTOS header files needed for your chosen board.
- Set up the necessary GPIO pins or other hardware components for your test.
- Make separate functions for each task that the program will handle.
- Create the tasks using FreeRTOS tools for task creation.
- Give each task a proper priority and enough memory (stack size) to work.
- Start the FreeRTOS scheduler so the tasks can run.
- Let each task do its job as planned.
- Use delays or blocking methods so tasks can take turns using the processor.
- Watch how the tasks run using LED flashes, serial messages, sensor data, or other hardware signs.
- Change the task priorities or delays and see how the task scheduling changes.
- Make sure multiple tasks can run smoothly without needing to be in one long loop.
- Write down what you saw about how the tasks worked and how the scheduler handled them.

CONCLUSION : The experiment shows how FreeRTOS allows a microcontroller with limited resources to handle multiple tasks at the same time.Instead of running all operations one after another in a single program, the application is split into separate tasks that the RTOS scheduler manages.

The experiment also explains what happens to tasks during different stages, like when they are waiting, running, or paused.
It shows how important it is to set priorities and timing for each task, as these factors determine when and how often a task gets to use the processor.This makes FreeRTOS a useful tool for building embedded systems where several actions need to happen at specific times and use the CPU efficiently.